

**Integración de JavaScript y Metodologías de Diseño Web en SIGAgro:
Buscador de Catálogo y Visor de Información Técnica**

Juan Carlos Abente y María Emilia Auras

Ingeniería en Sistemas de Información, Universidad de la Cuenca del Plata

Paradigmas y Lenguajes de Programación III, Comisión A

Ing. Dante Sicardi

30 de septiembre de 2026

Integración de JavaScript y Metodologías de Diseño Web en SIGAgro:

Buscador de Catálogo y Visor de Información Técnica

El presente trabajo documenta la incorporación de comportamiento en JavaScript a SIGAgro, un sistema de información geográfica agropecuaria destinado a la gestión de lotes y muestreos de un establecimiento hortícola de Andrade, Colonia Alemana, provincia de Misiones. El establecimiento cuenta con trece cultivos y alrededor de 93.000 plantas, distribuidas entre producción a campo, bajo invernadero e hidropónica.

La consigna de la cátedra requería enriquecer dos módulos del sitio desarrollado en la entrega anterior, cada uno según una tarjeta asignada, y fundamentar cada desarrollo con una metodología de diseño hipermedial (Cátedra de Paradigmas y Lenguajes de Programación III, 2026). Se trabajó sobre las opciones 3 y 8, cuyo alcance se resume en la Tabla 1.

Tabla 1

Tarjetas Desarrolladas

Opción	Módulo enriquecido	Metodología	Responsable
3. Buscador y filtro de catálogo	Listado de lotes, en tabla y en tarjetas	SOHDM	María Emilia Auras
8. Visor de información técnica	Mapa del establecimiento	VSDM	Juan Carlos Abente

El documento expone en primer lugar los criterios que guiaron la elección de las tarjetas. Luego describe el cumplimiento de los requisitos comunes, el desarrollo de cada opción y su correspondencia con el modelo relacional del sistema. Por último, presenta la fundamentación metodológica de ambos desarrollos.

Criterios de Elección de las Tarjetas

La elección no respondió a la dificultad técnica de cada tarjeta, sino al uso real que el ingeniero agrónomo hace del sistema. Se aplicaron cuatro criterios:

- **Centralidad.** Enriquecer pantallas existentes que concentran el uso cotidiano, en lugar de incorporar pantallas accesorias.

- **Coherencia con el modelo relacional.** Que los datos consumidos tengan correlato en la base de datos diseñada en la entrega anterior (Auras, 2026).
- **Sentido en el dominio.** Que la funcionalidad resuelva una tarea que el agrónomo efectivamente realiza.
- **Complementariedad.** Que ambas tarjetas operen sobre pantallas distintas y con metodologías distintas, sin superponerse.

Opción 3: El Listado como Pantalla de Trabajo

El listado de lotes es la pantalla donde el agrónomo identifica qué lotes tienen un cultivo determinado, cuáles se encuentran bajo invernadero o cuáles requieren intervención. En la entrega anterior la pantalla ya existía, pero sus filtros eran únicamente visuales. La tarjeta permitía volverlos funcionales sin rediseñar la interfaz.

Además, exigía trabajar con colecciones de datos mediante los métodos `filter`, `map`, `reduce` y `sort`, que constituyen la base de cualquier listado dinámico. La metodología SOHDM resultó adecuada porque parte de escenarios de uso, y la búsqueda constituye un escenario de exploración.

Opción 8: El Mapa como Núcleo del Sistema

En un sistema de información geográfica, la operación más habitual es seleccionar un elemento del mapa para consultar sus atributos. Hasta esta entrega, el mapa de SIGAgro solo mostraba los lotes coloreados según su estado, y seleccionar uno conducía a otra página.

La información técnica que el visor presenta ya se encontraba definida en el modelo relacional: campañas, observaciones con severidad y estado fenológico, plagas y malezas (Auras, 2026). Asimismo, la tabla de usuarios define tres roles, lo que otorga sustento a la distinción entre una vista pública y una vista técnica que plantea la tarjeta.

Opciones Evaluadas y Descartadas

Antes de la elección definitiva se evaluaron otras dos tarjetas. La Tabla 2 resume el motivo por el cual se descartó cada una.

Tabla 2*Opciones Evaluadas y Motivos de Descarte*

Opción	Pantalla	Motivo del descarte
5. Calculador de presupuesto	Solicitud de análisis	Se inició su desarrollo y se reemplazó por la opción 3. El cálculo de un presupuesto es una tarea periférica del sistema, mientras que la búsqueda de lotes es central.
12. Monitor de inventario y alerta	Mapa	Se planteó como un monitor de pérdida de plantas. Carecía de correlato en el modelo relacional, donde la cantidad de plantas es un valor fijo de la campaña. Tampoco tenía sentido agronómico: las pérdidas por plagas o clima son eventos masivos y no un stock que disminuye gradualmente.

Requisitos Transversales

La consigna establecía dos requisitos mínimos para todos los grupos: la captura desacoplada de eventos y la carga asíncrona de datos. La Tabla 3 detalla cómo los satisface cada desarrollo.

Tabla 3*Cumplimiento de los Requisitos Transversales*

Requisito	Opción 3	Opción 8
Eventos mediante <code>addEventListener</code> , sin atributos de evento en el HTML	Función <code>conectarEventos()</code> : input en el buscador, <code>change</code> en los desplegables y click en los encabezados.	Función <code>conectarEventos()</code> : click en lotes, botones de vista, menú y cierre; <code>change</code> en el rol; <code>keydown</code> para la tecla <code>Escape</code> .
Carga asíncrona con <code>fetch</code> , <code>async</code> y <code>await</code>	<code>cargarCatalogo()</code> solicita el catálogo una única vez, al cargar la página.	<code>obtenerLote()</code> solicita los datos de cada lote a demanda, al seleccionarlo.
Manejo de errores	Bloque <code>try/catch</code> y verificación del estado de la respuesta.	Igual, con un mensaje dentro del panel.

Se verifica la propiedad `ok` de la respuesta porque `fetch` no rechaza la promesa ante un código de error del servidor, sino solo ante una falla de red (Mozilla, s. f.-b). Dado que `fetch` no opera bajo el protocolo `file`, el sitio se sirve por HTTP durante el desarrollo.

La consigna menciona el archivo `app.js`. Ese nombre corresponde a la opción 3; la opción 8 se implementó en `visor.js`, puesto que cada página carga únicamente el

script que necesita.

Opción 3: Buscador y Filtro de Catálogo

El listado de lotes pasó de ser una maqueta con trece filas escritas en el HTML a cargarse desde un archivo de datos y filtrarse en el navegador mientras el usuario escribe, sin recargar la página. El desarrollo abarca las dos presentaciones del catálogo, cuyos archivos se describen en la Tabla 4.

Tabla 4

Archivos de la Opción 3

Archivo	Responsabilidad
datos/parcelas.json	Catálogo de los trece lotes. Única fuente de datos de ambas vistas.
js/app.js	Vista de tarjetas: búsqueda, filtros, ordenamiento e inyección.
js/tabla.js	Vista de tabla: un filtro por columna y ordenamiento por encabezado.
listado_box.html y listado_tabla.html	Estructura de cada vista, con el contenedor de resultados vacío.

Filtrado en Memoria

El catálogo se descarga una sola vez y se conserva en un arreglo que nunca se modifica. Cada filtrado aplica el método `filter`, que devuelve un arreglo nuevo con los elementos que cumplen las condiciones (ver Figura 1). En consecuencia, al vaciar la búsqueda reaparece el catálogo completo sin volver a solicitarlo.

Figura 1

Filtrado del Catálogo con Condiciones Acumulativas

```
let resultado = catalogo.filter(function (parcela) {
  const coincideTexto = texto === "" ||
    normalizar(parcela.nombre).includes(texto) ||
    normalizar(parcela.cultivo).includes(texto);
  const coincideSistema = sistema === "todos" || parcela.sistema === sistema;
  const coincideEstado = estado === "todos" || String(parcela.estado) === estado;
  return coincideTexto && coincideSistema && coincideEstado;
});
```

Nota. Fragmento de `app.js`. Cada condición contempla un valor neutro que no restringe el resultado.

Las condiciones se combinan mediante el operador lógico de conjunción, de modo que los filtros son acumulativos. El texto se compara normalizado, en minúsculas y sin tildes, de manera que la búsqueda “modulo” encuentra “Módulo hidropónico”.

Presentación y Ordenamiento

El resultado se transforma con `map` en un fragmento de marcado por lote, y los fragmentos se insertan en el contenedor. El recuento de plantas visibles se obtiene con `reduce`. Cuando ningún lote satisface los criterios, se muestra un mensaje en lugar de una grilla vacía.

En la vista de tabla, un clic sobre un encabezado ordena por esa columna y un segundo clic invierte el sentido. Los textos se comparan con `localeCompare` para respetar el orden alfabético del español, y el ordenamiento se aplica sobre una copia, dado que `sort` modifica el arreglo original.

Pruebas

La Tabla 5 resume los casos verificados y su resultado.

Tabla 5

Casos de Prueba de la Opción 3

Caso	Resultado obtenido
Carga inicial	Se muestran los trece lotes y el total de 93.000 plantas.
Búsqueda por cultivo	La búsqueda “tomate” devuelve los dos lotes correspondientes.
Búsqueda sin tildes	La búsqueda “modulo” encuentra “Módulo hidropónico”.
Filtros combinados	El cultivo y el sistema de producción se aplican a la vez.
Sin coincidencias	Se muestra el mensaje en lugar de un listado vacío.
Ordenamiento	Un clic ordena y un segundo clic invierte el sentido.
Ampliación del catálogo	Un lote agregado al archivo aparece en ambas vistas sin modificar el HTML.

Opción 8: Visor de Información Técnica

Al seleccionar un lote en el mapa, el visor consulta su información mediante `fetch` y la presenta en un panel lateral, con una vista general disponible para cualquier usuario y una vista técnica reservada a los roles que la requieren. Los archivos involucrados se describen en la Tabla 6.

Tabla 6

Archivos de la Opción 8

Archivo	Responsabilidad
js/visor.js	Consulta del detalle, construcción de las vistas y control del panel y de los roles.
css/visor.css	Panel lateral deslizante, escala de severidad y marca del lote seleccionado.
datos/lote/1.json a 13.json	Un archivo por lote, con los nombres de columna del modelo relacional.
index.html	Panel, identificador de cada polígono, botón de menú y carga del script.

Nota. Los estilos se ubicaron en una hoja propia para que ambos integrantes trabajaran en paralelo sin conflictos en el control de versiones.

Funcionamiento

Al seleccionar un lote se produce la siguiente secuencia:

1. El clic sobre el polígono, que es un enlace a la ficha del lote, se intercepta y `preventDefault` cancela la navegación.
2. Se marca el polígono seleccionado y se abre el panel lateral con un mensaje de carga.
3. Si el lote ya fue consultado, se toma de la memoria; de lo contrario, se solicita su archivo.
4. Si entretanto el usuario eligió otro lote, la respuesta anterior se descarta.
5. Se construye la vista general y, si el rol lo permite, la vista técnica.

El panel también se abre y se cierra con el botón de menú del mapa, que comunica su estado mediante los atributos `aria-controls` y `aria-expanded` (World Wide Web Consortium [W3C], 2023). Sin JavaScript, cada lote conserva su función de enlace a la

ficha: el visor mejora la página, pero no es imprescindible para navegarla.

Consulta Asíncrona con Memoria

La consulta se realiza a demanda: solo se descargan los datos del lote que el usuario selecciona, tal como funcionará con el servidor definitivo. Una estructura de memoria evita repetir solicitudes ya realizadas (ver Figura 2).

Figura 2

Consulta del Detalle de un Lote

```
async function obtenerLote(id) {
  if (cache.has(id)) return cache.get(id);

  const respuesta = await fetch(URL_LOTE(id));
  if (!respuesta.ok) {
    throw new Error("El servidor respondió " + respuesta.status);
  }
  const datos = await respuesta.json();
  cache.set(id, datos);
  return datos;
}
```

Nota. Fragmento de visor.js. La instrucción await espera la respuesta sin bloquear la interfaz.

Estado Sanitario Derivado

En el modelo relacional, el estado sanitario no constituye un atributo del lote: surge de sus observaciones (Auras, 2026). El visor lo calcula a partir de la severidad máxima registrada en el último muestreo, según la correspondencia de la Tabla 7. Los trece estados obtenidos coinciden con los que presentan los listados de la opción 3.

Tabla 7

Correspondencia entre Severidad y Estado Sanitario

Severidad máxima del último muestreo	Estado presentado
4 o 5	Requiere intervención
3	En seguimiento
1 o 2	Sin novedades
Sin campaña activa o sin muestreos	Sin campaña activa o sin muestreo

Captura de Eventos y Patrón Observer

El método `addEventListener` registra una función que el navegador ejecuta cuando ocurre un evento sobre un elemento (Mozilla, s. f.-a). Este mecanismo constituye una implementación del patrón Observer: el elemento actúa como sujeto, las funciones registradas como observadores, y el navegador las notifica cuando el evento se produce (Gamma et al., 1994). Como el sujeto desconoce a sus observadores, el HTML se limita a declarar la estructura y la lógica de comportamiento permanece en el script.

Pruebas

La Tabla 8 resume los casos verificados y su resultado.

Tabla 8

Casos de Prueba de la Opción 8

Caso	Resultado obtenido
Carga inicial	Panel cerrado; con perfil visitante, la vista técnica está deshabilitada.
Botón de menú	Abre el panel y desplaza los controles del mapa; un segundo clic lo cierra.
Selección del lote P-08	Se cancela la navegación, se abre el panel y se informa “Requiere intervención”.
Perfil agrónomo y vista técnica	Se muestran cinco observaciones, dos de ellas con severidad 4 por pulgón.
Lote ya consultado	No se genera una nueva solicitud.
Lote P-13	Se informa “Sin campaña activa” y la fecha de la última cosecha.
Cambio a perfil visitante	La vista regresa automáticamente a la general.
Tecla Escape y botón de cierre	Escape conserva la selección; el cierre la limpia.

Alineación con el Modelo Relacional

Los archivos de datos del visor reproducen la estructura del modelo relacional (Auras, 2026). Cada uno simula la respuesta de un futuro servicio que devuelva el detalle de un lote, de modo que, al migrar a Yii2 con MySQL, bastará con reemplazar la dirección consultada. La Tabla 9 muestra el origen de cada dato presentado.

Tabla 9*Correspondencia entre la Interfaz y el Modelo Relacional*

Dato en la interfaz	Origen en el modelo
Nombre y sistema de cultivo	lote.nombre, lote.sistema_cultivo
Cultivo y tipo	cultivo.nombre, cultivo.tipo
Plantas y fechas de la campaña	campana.cantidad_plantas, fecha_siembra, fecha_cosecha_estimada, fecha_cosecha_real, estado
Fenología y severidad	observacion.estado_fenologico, observacion.severidad
Plagas y malezas	observacion_plaga, observacion_maleza
Responsable de la observación	usuario: nombre, apellido y rol
Perfiles del selector	usuario.rol: técnico, agrónomo y administrador
Estado sanitario	No se almacena; se deriva de la severidad del último muestreo

El catálogo de la opción 3 se construyó antes de concluir el modelo relacional y conserva una estructura simplificada, con el sistema de cultivo expresado en etiquetas legibles y el estado sanitario precalculado. Del mismo modo, la interfaz de los listados emplea el término “parcela” donde el modelo utiliza “lote”. Ambas diferencias se unificarán en la etapa siguiente.

Fundamentación Metodológica

SOHDM en la Opción 3

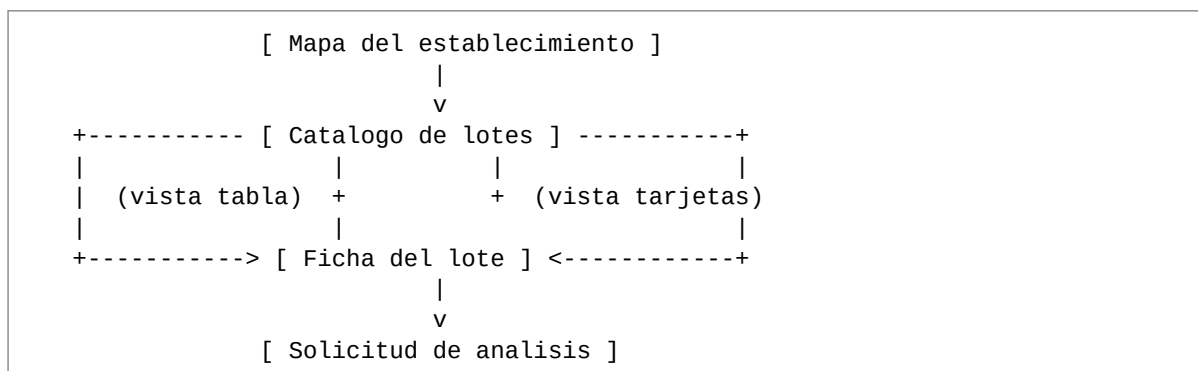
SOHDM (*Scenario-based Object-oriented Hypermedia Design Methodology*) se distingue por derivar el diseño a partir de los escenarios de uso y no del modelo conceptual del dominio (Lee et al., 1999). Desde esta perspectiva, el buscador no existe porque haya una entidad lote, sino porque existe un escenario concreto de exploración. La Tabla 10 resume la aplicación de cada fase.

Tabla 10*Aplicación de las Fases de SOHDM*

Fase	Aplicación en el módulo
Análisis mediante escenarios	Escenario de exploración y búsqueda de lotes.
Modelado de objetos	Lote, cultivo, sistema de producción, estado sanitario y muestreo.
Diseño de vistas	Resumen del lote, criterios de búsqueda y detalle.
Diseño navegacional	Catálogo como nodo índice con filtrado y enlace a la ficha.
Diseño de implementación	Dos presentaciones de una misma unidad de vista.
Construcción	Archivos app.js, tabla.js y parcelas.json.

Escenario de Exploración

El ingeniero agrónomo necesita identificar qué lotes presentan una condición determinada. Accede al catálogo, restringe el conjunto por texto, sistema de producción o estado sanitario, lo reordena, alterna entre tabla y tarjetas, e ingresa a la ficha del lote buscado. La Figura 3 representa el mapa navegacional resultante.

Figura 3*Mapa Navegacional del Escenario de Exploración*

Nota. Las vistas de tabla y de tarjetas son contextos de presentación de un mismo nodo.

La alternancia entre tabla y tarjetas no constituye una transición entre nodos, sino un cambio de contexto de presentación sobre el mismo nodo. El filtrado, a su vez, no genera nodos nuevos: reduce las anclas disponibles dentro del nodo índice. Esta distinción justifica resolverlo en el navegador en lugar de navegar hacia una página de resultados.

VSDM en la Opción 8

La tarjeta solicita definir la visibilidad del sistema mediante una vista pública y una vista técnica (Cátedra de Paradigmas y Lenguajes de Programación III, 2026). El enfoque es afín a los métodos orientados a la audiencia, que clasifican primero a los usuarios y diseñan luego la información que corresponde a cada clase (De Troyer y Leune, 1998). El principio aplicado es que un mismo dato se presenta de manera distinta según quién lo consulta: cada lote tiene una única fuente y lo que varía es la proyección mostrada.

Perfiles de Usuario

Los perfiles se toman directamente de la tabla de usuarios del modelo, como muestra la Tabla 11.

Tabla 11

Perfiles de Usuario y Vista Asignada

Perfil	Rol en el modelo	Vista	Necesidad
Visitante	Sin sesión	Pública	Saber si un lote requiere atención.
Técnico	tecnico	Técnica	Consultar el detalle de las observaciones.
Ingeniero agrónomo	agronomo	Técnica	Decidir intervenciones según la severidad.
Administrador	administrador	Técnica	Acceso completo al sistema.

Matriz de Visibilidad

La Tabla 12 indica qué atributos componen cada vista.

Tabla 12*Matriz de Visibilidad por Vista*

Atributo	Vista pública	Vista técnica
Cultivo, sistema, superficie y plantas	Sí	Sí
Fechas de siembra y de cosecha, días a cosecha	Sí	Sí
Estado sanitario y fecha del último relevamiento	Sí	Sí
Tipo de cultivo y estado fenológico	No	Sí
Días desde la siembra y densidad de plantación	No	Sí
Puntos de muestreo y severidad máxima	No	Sí
Historial de observaciones: plagas, malezas, notas y responsable	No	Sí

Transición entre Vistas

Pasar de una vista a otra no constituye una navegación: el nodo es el mismo y cambia la proyección presentada. Con perfil visitante la vista técnica permanece deshabilitada, y si el perfil cambia a uno sin permiso, la interfaz regresa automáticamente a la vista pública.

Criterio de Seguridad

En este prototipo, el control de acceso es visual: el archivo de datos llega completo al navegador. En la versión productiva, el servidor no enviará la sección técnica a un perfil no autorizado, mediante el control de acceso por roles del framework. Ocultar información en la interfaz no constituye, por sí solo, una medida de seguridad.

Complementariedad de Ambas Metodologías

Las dos fundamentaciones responden preguntas distintas sobre el mismo sistema. SOHDM define qué se explora y cómo se navega, a partir de lo que el usuario necesita hacer. VSDM define qué parte de esa información corresponde a cada perfil. En conjunto, cubren la navegación y la visibilidad del sistema.

Conclusiones

La incorporación de JavaScript transformó una maqueta estática en una interfaz que responde a las necesidades del usuario. El cambio más relevante no es visual sino estructural: los datos dejaron de estar escritos en el marcado y pasaron a constituir una fuente independiente. Incorporar un lote ya no exige modificar varias páginas, sino un archivo de datos.

Ese desacoplamiento prepara la etapa siguiente del proyecto. Cuando el sistema se construya sobre Yii2 y MySQL, los archivos de datos serán reemplazados por consultas a la base, mientras que la lógica de búsqueda, presentación y visibilidad se mantendrá.

Desde el punto de vista metodológico, las decisiones de diseño quedaron fundamentadas: resolver el filtrado dentro del mismo nodo, tratar la alternancia de vistas como un cambio de presentación y derivar la visibilidad de los roles del modelo. También lo quedó la elección de las tarjetas, orientada por el uso real del sistema y por su coherencia con la base de datos.

El código fuente se encuentra disponible en el repositorio del proyecto:

<https://github.com/juankiabente/sigagro>

Referencias

- Auras, M. E. (2026). *AE2: Modelo relacional de SIGAgro* [Trabajo práctico no publicado]. Universidad de la Cuenca del Plata.
- Cátedra de Paradigmas y Lenguajes de Programación III. (2026). *Reto: Startup relámpago. Integración JS y metodologías web* [Consigna de trabajo práctico]. Universidad de la Cuenca del Plata.
- De Troyer, O. M. F., y Leune, C. J. (1998). WSDM: A user centered design method for Web sites. *Computer Networks and ISDN Systems*, 30(1–7), 85–94.
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Lee, H., Lee, C., y Yoo, C. (1999). A scenario-based object-oriented hypermedia design methodology. *Information & Management*, 36(3), 121–138.
- Mozilla. (s. f.-a). *EventTarget: addEventListener() method*. MDN Web Docs. Recuperado el 30 de septiembre de 2026, de <https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener>
- Mozilla. (s. f.-b). *Using the Fetch API*. MDN Web Docs. Recuperado el 30 de septiembre de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- World Wide Web Consortium. (2023). *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. <https://www.w3.org/TR/wai-aria-1.2/>